



Animales

Nombres y apellidos del alumno(a):
Joshua Ontiveros Cabrero

Nombre del docente:
Naranjo Avilez Jesus

Carrera Tecnologías de la información:
Programación orientada a objetos.

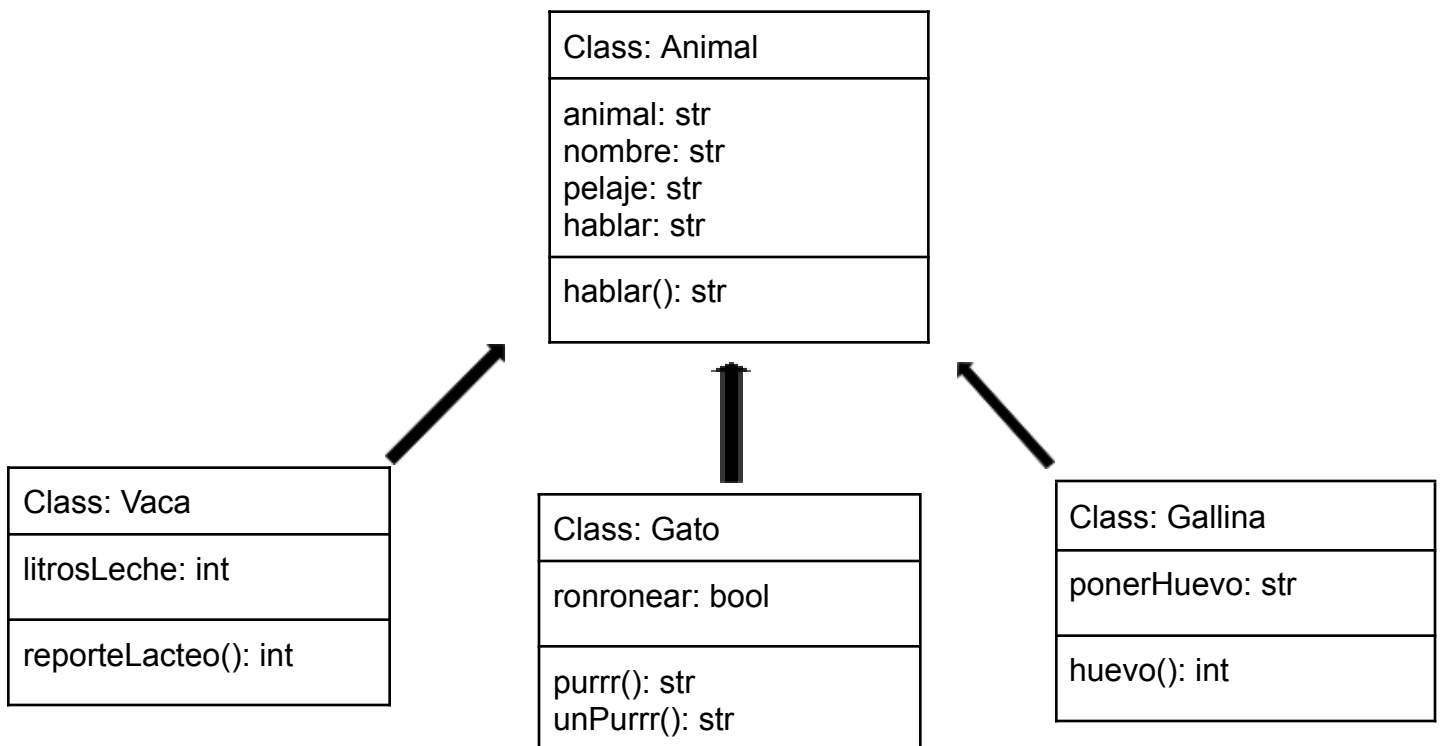
Universidad Politécnica de Baja California.
Mexicali Baja California

29/05/2026

Introduccion:

En esta actividad recreamos de una manera un poco distinta la anterior actividad, usamos esa misma estructura la cual utilizamos para hacer el programa de los vehículos y sus variantes tales como el carro, la moto o un camión de carga, con esa misma estructura nos enfocamos en el reino animal, usando como la clase padre el concepto de animal y luego en las clases hijas tenemos a un gato, una gallina y una vaca un tanto peculiar.

Quizás puedan divergir bastante estos 2 conceptos pero en esta materia podemos simplificarlo de forma que podremos procesar mejor esta información. En mi mapa UML la clase padre es Animales, el concepto general de animales es quien albergara las cosas mas generales que tienen en comun, como un nombre, una raza, un sonido y tipo de pelaje, de ahí las clases hijas tienen cosas mas especificas de cada especie, una gallina con plumas y poner huevos, una vaca que genera leche y un gato que ronronea.



Desarrollo:

Primero que nada creamos nuestro archivo “animales.py” y comenzamos importando las librerías de abc (Abstract Base Class) y la marcamos con Abstract method, para resguardar con mayor seguridad lo que contenga la clase padre; tras eso definiremos la clase “Animal” que será la clase padre.

 animales.py

```
#animales.py  
  
from abc import ABC, abstractmethod
```

```
class Animal(ABC)
```

Acto seguido instanciamos todos los atributos que inherentemente todos los animales tienen en comun, su especie animal, su nombre, su pelaje o la capacidad de comunicarse.

```
def __init__(self, animal, pelaje, nombre, hablar):  
    self.__animal = animal  
    self.__nombre = nombre  
    self.__pelaje = pelaje  
    self.__hablar = hablar
```

Después deberemos definir la función para poder hablar, usando el return para que devuelva un valor y lo envíe, la segunda funcion utilizada es obtener nombre, mas a futuro indagaremos en ella.

```
def hablar(self):  
    return print(self.__hablar)  
  
def obtener_nombre(self):  
    return self.__nombre
```

Para cerrar con nuestro “animal.py” definimos obtener datos, el cual nos enviará la información general de los animales, este dato se verá afectado dependiendo de que animal es el que manda la información y además, para que tenga coherencia el texto se añadió 1 condición, si el animal es una gallina el texto dirá plumaje en lugar de pelaje, todo con el fin de mantener la coherencia.

```
def obtenerDatos(self) -> str:
    if self.__animal == 'Gallina':
        return f"Animal {self.__animal} de plumaje {self.__pelaje} y nombre {self.__nombre}."
    else:
        return f"Animal {self.__animal} de pelaje {self.__pelaje} y nombre {self.__nombre}."
```

De ahí tendríamos que crear una de las clases hijas siendo estas los animales, en mi caso desee comenzar con un gato

 gato.py

Primero lo primero, importamos desde animales nuestra clase animal para poder utilizarla en nuestro archivo.

```
from animales import Animal
```

Acto seguido crearemos la clase Gato con herencia de Animal, esto con el fin de utilizar lo que hemos instanciado en animal. Instanciamos con el __init__ todo aquello que utilizara el gato, añadiendo ahora el atributo de ronronear, utilizamos “super” para enviar nuestra información y que sea procesada. Después definiremos como atributo exclusivo de nuestro gato la capacidad de ronronear, este comienza en silencio por defecto.

```
class Gato (Animal):
    def __init__(self, animal, pelaje, nombre, hablar, ronronear):
        super().__init__(animal, pelaje, nombre, hablar)

        self.__ronronear = 'silencio'
```

Definiremos ahora las funciones de nuestro gato, definimos purr como la accion de ronronear y usaremos puertas logicas para poder determinar si puede comenzar a ronronear, no deberia comenzar a ronronear si ya esta ronroneando, por el contrario definimos Un_purr como lo opuesto, el parar de ronronear, y como en el caso anterior, no podemos parar de ronronear si no estaba ronroneando desde un principio.

```
def purr(self):
    print("*ronroneando*")
    if self.__ronronear == 'silencio':
        print("purroooooooooooooooooooooooooooo")
        self.__ronronear = 'ruido'
    else:
        print("Ya estoy ronroneando!")

def Un_purr(self):
    print("*desronroneando*")

    if self.__ronronear == 'ruido':
        print(".....")
        self.__ronronear = 'silencio'

    else:
        print("Ya estoy en silencio!")
```

Haremos exactamente lo mismo con las 2 clases restantes, primero usando de ejemplo a la gallina, que tras importar de animales la clase Animal definimos la clase Gallina como nuestro animal, en el inicializador instanciamos toda su informacion, mas sin embargo al final definimos “ponerHuevo”, despues definimos el valor de ponerHuevo, puesto que sera como un contador para nosotros, llevara la cuenta de cuantos huevos se han puesto a la par que confirme que en efecto, se puso un huevo. Definimos la funcion de huevo para que se ejecute la accion de poner huevos y tambien hacer la cuenta y sumarla

```
#gallina.py

from animales import Animal

class Gallina (Animal):
    def __init__(self, animal, pelaje, nombre, hablar, ponerHuevo):
        super().__init__(animal, pelaje, nombre, hablar)

        self.__ponerHuevo = 0

    def huevo(self):
        print("Poniendo un huevo")

        self.__ponerHuevo += 1

        print(f"Contador de huevos: {self.__ponerHuevo}")
```

En la siguiente instancia que es nuestra Vaca, importaremos de los animales la clase Animal, acto seguido definimos la clase Vaca con herencia de Animal, definimos nuestro inicializador añadiendo el atributo de litrosLeche la cual servira como un contador de cuanta leche se ha acumulado y confirmador de que se realizo, de igual manera se usan puertas logicas para determinar que no supere el limite.

```
#vaca.py

from animales import Animal

class Vaca (Animal):
    def __init__(self, animal, pelaje, nombre, hablar, litrosLeche):

        super().__init__(animal, pelaje, nombre, hablar)

        self.__litrosLeche = 0

    def reporteProduccion(self):
        if self.__litrosLeche < 21:
            print("Produciendo leche")
            self.__litrosLeche += 1
            print(f"Reporte lacteo: {self.__litrosLeche} litro(s) producido(s).")
        else:
            print(f"{self.obtener_nombre()} alcanzo su limite maximo.")
```

Ya para poder hacer utilizable nuestro codigo necesitamos crear el main que sera la goma de nuestro programa, para poder utilizar todo lo que hemos creado necesitamos importar desde los otros archivos la informacion, importaremos del archivo gato a la clase Gato y lo mismo con los demas animales.

```
from gato import Gato
from gallina import Gallina
from vaca import Vaca
```

Definimos nuestro main que va a instanciar nuestro programa

```
def main():
    print("Instanciando.....")
```

Despues definiremos mi_gato, el cual mandara la informacion a nuestro Animales que pondra en esos espacios que dejamos para mostrar el tipo de animal, pelaje, etc.

```
mi_gato = Gato("Gato", "Naranja", "Jibanyan", "Meow :3",str)
```

Ya para poder mostrar que funciona todo lo que hemos hecho tenemos que hacer que el programa llame a las funciones que hemos creado para ver si surgen efecto, primero lo etiquetamos como metodos felinos para hacerle separacion, despues usamos print(mi_gato.. Etc. para poder mostrar los datos que hemos mandado, acto seguido printeamos un “el gato hace” para que conecte con la funcion “hablar” todo con fines de coherencia.

Tras eso podemos probar la funcion de ronronear, y desronronear para ver si funciona

```
print("Metodos felinos")
print(mi_gato.obtenerDatos())
print("El gato hace: ", end=" ")
mi_gato.hablar()
mi_gato.purr()
mi_gato.Un_purr()
print("\r")
```

Haremos exactamente lo mismo con mi_gallina y mi_vaca

```
mi_gallina = Gallina("Gallina", "Blanco", "Turuleca", "Pock-Pock :^", int)
print("Metodos aviarios")
print(mi_gallina.obtenerDatos())
print("La gallina hace: ",end=" ")
mi_gallina.hablar()
mi_gallina.huevo()
print("\r")

mi_vaca = Vaca("Vaca", "Blanco", "Conquest", "Mooo (I am... so lonely, i dont even have a name, just a purpose).", int)
print("Metodos bovinos")
print(mi_vaca.obtenerDatos())
print("La vaca hace: ",end=" ")
mi_vaca.hablar()
mi_vaca.reporteProduccion()
print("\r")
```

Para cerrar, pondremos esto al final para que pueda funcionar nuestro codigo, ahora lo ejecutamos

```
if __name__ == "__main__":
    main()
```

Aquí podemos ver cada instancia, funcion y clase trabajando correctamente como es debido, mostrando el tipo de animal, pelaje y nombre, su sonido y su funcion especifica, dependiendo el animal, el gato ronronea, la gallina pone huevos y la vaca hace leche.

```
Instanciando.....
Metodos felinos
Animal Gato de pelaje Naranja y nombre Jibanyan.
El gato hace: Meow :3
*ronroneando*
purrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrr
*desronroneando*
.....

Metodos aviarios
Animal Gallina de plumaje Blanco y nombre Turuleca.
La gallina hace: Pock-Pock :^
Poniendo un huevo
Contador de huevos: 1

Metodos bovinos
Animal Vaca de pelaje Blanco y nombre Conquest.
La vaca hace: Moo (I am... so lonely, i dont even have a name, just a purpose).
Produciendo leche
Reporte lacteo: 1 litro(s) producido(s).
```

Conclusion:

El programa fue un lavado de cara a el programa anterior puesto que se uso casi que la misma estructura mas sin embargo el hecho de darle un enfoque a animales la vuelve una actividad un poco mas intuitiva con datos que muchas más personas podrían comprender, gracias a esta actividad pude resolver una incógnita que tenía puesto que cada programa que hacia mostraba un valor "None" al final de cada acción, logre resolverlo y me siento contento con haberlo hecho.

Dejando eso de lado, con cada actividad que hacemos siento que comprendo mejor la programacion orientada a objetos y me esta gustando, esto de hacer todo modular y en pequeños segmentos que se entrelazan entre sí, da para hacer un proyecto mas grande y con lo que se avecina del proyecto integrador me siento con emocion de ver como resultara..